ПР1. Создание NLP-пайплайна для обработки текстовых данных

1/ Цель работы

Разработать модульный, воспроизводимый конвейер (пайплайн) предобработки текста на Python, который очищает сырой текст, приводит его к нормальной форме и готовит для дальнейшего анализа.

Зачем это нужно?
Нельзя подавать грязные данные в модель - это аксиома Computer Science. Опечатки и разные словоформы (например, «бежать», «бегу», «бежал») математически воспринимаются моделью как разные слова, что увеличивает разреженность данных и снижает точность. Мы должны понять, что текст - это всего лишь последовательность токенов, и научиться управлять этой последовательностью.

Легенда
Вы занимаетесь анализом новостного трафика или мониторингом отзывов. Текстовые данные поступают в сыром виде, они зашумлены, содержат несвязанные символы и различную грамматику. Прежде чем строить модели машинного обучения или проводить аналитику, данные нужно превратить в чистый, структурированный формат.

Входные данные:

- датасет `dataset.csv` (или текстовый файл), содержащий несколько сотен новостных заголовков или отзывов (*в данной работе приводится пример с отзывами*)

Выходные данные:

- очищенный текст, визуализация статистики, отчет о работе.

2/ Краткая теория

Текст - это неструктурированные данные. Для компьютера текст - это просто строка (байты). Чтобы алгоритмы машинного обучения (которые работают с числами) поняли текст, мы должны разбить его на атомарные единицы и привести к единому знаменателю.

2.1/ Этапы конвейера обработки текста

1. **Очистка (Cleaning)**
   Удаление "шума", не несущего смысловой нагрузки.
   Объекты: HTML-теги, URL-адреса, спецсимволы (`@#$`), лишние пробелы. Важно действовать аккуратно: удаление знаков препинания (пунктуации) спорно. В классическом подходе мы удаляем пунктуацию, чтобы слова типа "привет," и "привет" не считались разными токенами.
   Метод: Регулярные выражения (regex).

2. Токенизация (Tokenization)
Разбиение потока текста на минимальные единицы — токены (слова, числа, знаки препинания).
Виды:

- *Word tokenization:* Разбивка по пробелам и пунктуации.
- *Sentence tokenization:* Разбивка на предложения.

3. Нормализация

- **Лемматизация (Lemmatization):** Приведение слова к его канонической (словарной) форме с учетом морфологии.
  - *Пример:* `лучшими` -> `хороший`, `бежал` -> `бежать`.
  - *Требует:* Морфологического словаря и части речи.
- **Стемминг (Stemming):** "Отсечение" окончаний. Более грубый, быстрый, но не всегда лингвистически верный.
  - *Пример:* `лучшими` -> `лучш`. Слово может потерять смысл.

(*В нашей работе используем лемматизацию для максимальной чистоты.*)

4. Удаление стоп-слов (Stop Words Removal)
Удаление часто встречающихся слов, не несущих смысловой нагрузки для анализа (предлоги, союзы, частицы: "и", "в", "на", "но").

2.2/ Библиотеки

NLTK (Natural Language Toolkit) — это одна из самых известных и старых библиотек для обработки естественного языка (NLP) в Python.

Основные возможности:

- **Предобработка текста:** токенизация (разбиение на слова/предложения), удаление стоп-слов.
- **Морфологический анализ:** стемминг (отсечение окончаний), лемматизация (приведение к начальной форме).
- **Синтаксический анализ:** POS-тегирование (части речи), парсинг (деревья зависимостей).
- **Работа с данными:** доступ к большим коллекциям текстов (корпусам) и словарям.

---

spaCy — это современная, высокопроизводительная библиотека для обработки естественного языка (NLP) на Python, ориентированная на практическое применение в продакшене.

Основные возможности:

- **Быстрая предобработка:** эффективная токенизация, нормализация, работа с юникодом.
- **Лингвистический анализ:** POS-тегирование, синтаксический парсинг (зависимости), распознавание именованных сущностей (NER).
- **Векторизация и семантика:** встроенные word vectors, вычисление семантической схожести токенов и документов.

- **Конвейеры (pipelines):** модульная архитектура — можно легко добавлять, отключать или заменять компоненты обработки.
- **Поддержка множества языков:** готовые модели для английского, немецкого, французского, испанского, португальского, китайского и др. (включая экспериментальную поддержку русского).

Ход работы

Шаг 1. Подготовка среды и данных

1. Создайте виртуальное окружение Python
2. Установите необходимые библиотеки

```
pip install nltk spacy pandas matplotlib wordcloud
```

3. Загрузите языковые модели (RU и EN)

```
python -m spacy download ru_core_news_sm
python -m spacy download en_core_web_sm
```

4. Загрузите dataset. Предлагается список отзывов с Кинопоиска: https://huggingface.co/datasets/zloelias/kinopoisk-reviews-short
   Загрузка этого датасета с помощью Pandas:

```
import pandas as pd

splits = {'train': 'data/train-00000-of-00001.parquet', 'test': 'data/test-00000-of-00001.parquet'}
df = pd.read_parquet("hf://datasets/zloelias/kinopoisk-reviews-short/" + splits["train"])
```

(*этот код взят со страницы датасета по ссылке выше*)

> **Примечание**
>
> Если на этом моменте будут ошибки, обновите пакеты:
>
> ```
> pip install --upgrade pandas pyarrow huggingface_hub fsspec
> ```

Шаг 2. Исследовательский анализ данных (EDA) сырого текста

Перед очисткой текста посмотрите на исходные данные (с чем имеете дело), сделайте некоторые оценки текста.

1. Выведите 10 случайных отзывов (строк в таблице)
2. Посчитайте общее количество символов и примерное количество слов (по пробелам)
3. *Вопрос:* Есть ли явный мусор (ссылки, цифры)?

Шаг 3. Создание класса-обработчика

Почему класс? Потому что мы хотим переиспользовать пайплайн.

Создайте класс `TextPreprocessor`.
В конструктор класса (`__init__`) передавайте:

- `language` ('ru' или 'en')
- `remove_stopwords` (True/False)
- `use_lemmatization` (True/False) — для сравнения результатов.

Внутри конструктора загружайте модели NLTK и spaCy один раз, чтобы не тратить время на каждой итерации.

Шаг 4. Реализация методов очистки

Реализуйте метод `_clean_text(raw_text: str) -> str`.

- Приведите текст к нижнему регистру (`lower()`).
- Удалите URL (используйте `re.sub(r'http\S+', '', text)`).
- Удалите HTML теги (`re.sub(r'<[^>]+>', '', text)`).
- Удалите лишние символы (оставьте только буквы и пробелы). *Осторожно:* для русского языка важно сохранять кириллицу.

```
text = re.sub(r'[^a-zA-Za-яA-ЯёЁ\s]', '', text)
```

- Замените множественные пробелы на один.

Шаг 5. Интеграция NLTK и spaCy

Создайте основной метод `process(text: str) -> list`.

1. **Токенизация:**
   - Если используем NLTK: `nltk.word_tokenize(cleaned_text)`.
   - Если используем spaCy: `doc = nlp(cleaned_text)` — спайси-документ уже является итератором по токенам.
2. **Лемматизация и фильтрация:**
   - Пройдите по токенам.
   - Для spaCy это `token.lemma_`, для NLTK потребуется дополнительная магия с частями речи (сами ищите как...).
   - Удалите стоп-слова (проверьте `token.is_stop` в spaCy или `token in nltk.corpus.stopwords`).
   - Удалите токены-пунктуацию (в spaCy это `token.is_punct`).

Шаг 6. Создание конвейера (пайплайна)

Создайте функцию `pipeline(texts: list, processor: TextPreprocessor) -> pd.DataFrame`.

- На вход подается список сырых текстов.

- Функция возвращает DataFrame с колонками: `raw_text`, `clean_text` (результат шага 3), `tokens` (результат шага 4).
- **Опционально.** Используйте `tqdm` для отображения прогресс-бара

Шаг 7. Визуализация статистики

Постройте графики:

1. **Гистограмма распределения длины текстов (до и после очистки).** Показывает, насколько сильно чистка сжимает текст.
2. **Облако слов (WordCloud).** Постройте для лемматизированных токенов (без стоп-слов). Это визуализирует смысл корпуса.
3. **Топ-20 частотных слов.** Постройте барплот.

Форма отчета

1. Предоставить код (класс-обработчик)
2. Сохраните очищенный DataFrame в CSV
3. Визуализация статистики